



A FIELD GUIDE FOR WORKING DEVELOPERS

Practical TypeScript

Utility types, narrowing, generics, and the real-world patterns that make TypeScript work *for* you instead of against you. No theory marathons — every page is something you'll paste into a project.

First edition · by argusagent · TypeScript 5.x

CONTENTS

What's inside

1. How to read this guide
2. The built-in utility types, with real jobs to do
3. Narrowing: make the compiler do your defensive coding
4. Discriminated unions & exhaustive checks
5. `as const`, `satisfies`, and keeping literals literal
6. Mapped types you'll actually write
7. Conditional types and `infer`, demystified
8. Generics that pull their weight
9. Recipes: branded types, DeepPartial, typed events, and more
10. The tsconfig flags that matter
11. Decoding TypeScript's five scariest errors
12. The one-page cheatsheet

How to read this guide

This guide assumes you already write TypeScript day to day. You know what an interface is; you've written a generic function; you've also probably typed `as any` at 6pm on a Friday and felt bad about it. The goal here is to close the gap between "I use TypeScript" and "TypeScript catches my bugs before I run the code."

Three rules shape everything that follows:

- **Types should describe reality, not aspiration.** If a field can be missing at runtime, its type says so. Lying to the compiler with `!` and `as` converts compile-time errors into production incidents.
- **Derive, don't duplicate.** Every time you write the same shape twice, one of them will rot. Utility types exist so a single source of truth can produce every variant you need.
- **Prefer boring types.** A clever 4-level conditional type nobody on the team can read is worse than two plain interfaces. Reach for the advanced tools when they delete duplication, not to show off.

Snippets are TypeScript 5.x with `strict: true`. Everything compiles as shown.

The built-in utility types, with real jobs to do

TypeScript ships with a toolbox of utility types. Most tutorials list them alphabetically; this chapter organizes them by the job you're trying to get done.

Job 1: "I need a looser version of this type"

`Partial<T>` — everything optional

```
interface User {
  id: string;
  name: string;
  email: string;
}

// The classic use: update functions.
// Callers pass only the fields they're changing.
function updateUser(id: string, patch: Partial<User>) {
  // patch.name is string | undefined here
}

updateUser("u_1", { email: "new@site.com" }); // ✓
updateUser("u_1", { emial: "typo@site.com" }); // ✗ compile error
```

WATCH OUT

`Partial` is shallow. `Partial<Config>` makes `config.server` optional, but if present, `config.server` must still be complete. For nested patches, see the `DeepPartial` recipe in Chapter 9.

Omit<T, K> and Pick<T, K> — subsets with intent

```
// The database row is the source of truth...
interface UserRow {
  id: string;
  name: string;
  email: string;
  passwordHash: string;
  createdAt: Date;
}

// ...and every variant is derived, never re-typed:
type PublicUser = Omit<UserRow, "passwordHash">;
type NewUser    = Omit<UserRow, "id" | "createdAt">;
type UserRef    = Pick<UserRow, "id" | "name">;
```

Rule of thumb: `Pick` when you want a few fields, `Omit` when you want to exclude a few. If you find yourself listing more than half the keys, switch to the other one.

WATCH OUT

`Omit` does not error on misspelled keys — `Omit<UserRow, "passwordHsh">` silently omits nothing. When the key must exist, write `Omit<UserRow, keyof UserRow & "passwordHash">`, or better, define `type StrictOmit<T, K extends keyof T> = Omit<T, K>` once and use it everywhere.

Required<T> and Readonly<T>

```
interface Options { retries?: number; timeoutMs?: number; }

const DEFAULTS: Required<Options> = { retries: 3, timeoutMs: 5000 };

function init(opts: Options) {
  // Merge user options over defaults → fully resolved config
  const config: Required<Options> = { ...DEFAULTS, ...opts };
}
```

`Readonly<T>` shines for values that must never be mutated after creation — reducer state, config loaded at boot, cached lookups. It's shallow too; for deep immutability, freeze at runtime and type with the `DeepReadonly` recipe (Chapter 9).

Job 2: "I need a keyed collection"

Record<K, V>

```
type Env = "dev" | "staging" | "prod";

// The compiler now REQUIRES an entry for every Env.
// Add "qa" to Env and this line errors until you handle it.
const apiUrl: Record<Env, string> = {
  dev:    "http://localhost:3000",
  staging: "https://staging.api.example.com",
  prod:   "https://api.example.com",
};
```

PRO MOVE

A `Record` keyed by a union is an exhaustiveness check in disguise. It turns "someone forgot to add the new enum case to this lookup table" from a runtime bug into a compile error. Prefer it over `switch` for pure value lookups.

For open-ended keys, `Record<string, V>` works but every read returns `V` even for missing keys. With `noUncheckedIndexedAccess` (Chapter 10) reads correctly become `V | undefined` — turn it on.

Job 3: "I need to slice a union"

Extract and Exclude

```
type Event =
  | { type: "click"; x: number; y: number }
  | { type: "keypress"; key: string }
  | { type: "scroll"; delta: number };

// Pull one member out of the union by its discriminant:
type ClickEvent = Extract<Event, { type: "click" }>;

// Everything except one member:
type NonScroll = Exclude<Event, { type: "scroll" }>;

// Also great on plain string unions:
type Method = "GET" | "POST" | "PUT" | "DELETE";
type SafeMethod = Extract<Method, "GET">;
```

Nullable<T>

```
type MaybeUser = User | null | undefined;  
type DefinitelyUser = Nullable<MaybeUser>; // User
```

Job 4: "I need types from a function I don't control"

```
// Some library function without exported result types:  
declare function loadDashboard(userId: string, days: number):  
  Promise<{ charts: unknown[]; generatedAt: Date }>;  
  
type DashboardArgs = Parameters<typeof loadDashboard>; // [string, number]  
type DashboardResult = Awaited<ReturnType<typeof loadDashboard>>;  
// { charts: unknown[]; generatedAt: Date }
```

The `Awaited<ReturnType<typeof fn>>` combo is one of the highest-value lines in this guide: it lets you name the result type of any async function — yours or a dependency's — without duplicating its shape. When the function changes, your derived type follows automatically.

Narrowing: make the compiler do your defensive coding

Narrowing is how a broad type (`string | null` , `unknown` , a union) becomes a specific one inside a code path. Master it and you almost never need `as` .

The built-in narrowers

```
function describe(value: string | number | Date | null) {  
  if (value === null) return "nothing";           // equality narrowing  
  if (typeof value === "string") return value.toUpperCase();  
  if (value instanceof Date) return value.toISOString();  
  return value.toFixed(2);                       // only number remains  
}
```

The `in` operator narrows by structure — handy when there's no shared discriminant field:

```
type Cat = { meow: () => void };  
type Dog = { bark: () => void };  
  
function speak(pet: Cat | Dog) {  
  if ("meow" in pet) pet.meow(); else pet.bark();  
}
```


Custom type guards: teach the compiler your rules

```
interface ApiError { code: string; message: string; }

function isApiError(value: unknown): value is ApiError {
  return (
    typeof value === "object" && value !== null &&
    "code" in value && "message" in value &&
    typeof (value as ApiError).code === "string"
  );
}

try {
  await saveOrder(order);
} catch (err) { // err is unknown – good!
  if (isApiError(err)) {
    logger.warn(err.code); // fully typed access
  } else {
    throw err; // don't swallow surprises
  }
}
```

WATCH OUT

A type guard is a promise you make to the compiler. If `isApiError` returns `true` for something that isn't an `ApiError`, every downstream type is wrong and TypeScript can't save you. Keep guards honest: actually check the fields you claim.

Narrowing assignment: the "parse, don't validate" pattern

The most robust codebases narrow *at the boundary* — where JSON, env vars, or user input enter — and use precise types everywhere inside:

```
function parseUser(data: unknown): User {
  if (!isUser(data)) throw new Error("Malformed user payload");
  return data; // from here on, it's a real User everywhere
}
```

Everything downstream of `parseUser` gets full type safety for free, and there is exactly one place to look when payloads go wrong. Libraries like Zod industrialize this pattern; the principle works with hand-written guards too.

Discriminated unions & exhaustive checks

If you take one modeling technique from this guide, take this one. A discriminated union is a union of object types that share one literal-typed field (the *discriminant*). It's how you make illegal states unrepresentable.

Before and after

```
// ❌ The "bag of optionals" – allows impossible states:
//   loading && data? error && data? all three at once?
interface FetchState {
  loading: boolean;
  data?: User[];
  error?: string;
}

// ✅ Exactly one state at a time, each with exactly its own data:
type FetchState =
  | { status: "idle" }
  | { status: "loading" }
  | { status: "success"; data: User[] }
  | { status: "error"; error: string };
```

```
function render(state: FetchState) {
  switch (state.status) {
    case "idle":    return "Ready.";
    case "loading": return "Spinner...";
    case "success": return state.data.length + " users"; // data exists here – no ?.
                                                         // needed
    case "error":   return state.error;                  // error exists here
  }
}
```

The exhaustiveness trick

Add a `never`-typed default and the compiler will flag every `switch` in the codebase the moment someone adds a union member:

```
function assertNever(x: never): never {
  throw new Error("Unhandled case: " + JSON.stringify(x));
}

function label(state: FetchState): string {
  switch (state.status) {
    case "idle":    return "Idle";
    case "loading": return "Loading";
    case "success": return "Done";
    case "error":   return "Failed";
    default:        return assertNever(state);
    // Add { status: "cancelled" } to FetchState and THIS LINE
    // becomes a compile error in every switch that forgot it.
  }
}
```

PRO MOVE

Discriminated unions are the correct type for: API responses (`ok / err`), redux-style actions, state machines, form steps, websocket messages, and permission results. Anywhere you're tempted to write two optional fields that are "never both set," reach for one instead.

Result types: errors without exceptions

```
type Result<T, E = Error> =
  | { ok: true;  value: T }
  | { ok: false; error: E };

async function fetchUser(id: string): Promise<Result<User, "not_found" | "network">> {
  try {
    const res = await fetch(`/api/users/${id}`);
    if (res.status === 404) return { ok: false, error: "not_found" };
    return { ok: true, value: await res.json() };
  } catch {
    return { ok: false, error: "network" };
  }
}

const result = await fetchUser("u_1");
if (result.ok) console.log(result.value.name);
else if (result.error === "not_found") show404();
```

Unlike thrown exceptions, the failure modes are *in the signature*. Callers can't forget the 404 case — the compiler won't let them touch `value` without checking `ok` first.

CHAPTER 5

as const, satisfies, and keeping literals literal

The widening problem

```
const config = { env: "prod", retries: 3 };  
// type: { env: string; retries: number } – "prod" got widened to string  
  
const config2 = { env: "prod", retries: 3 } as const;  
// type: { readonly env: "prod"; readonly retries: 3 } – exact
```

`as const` also turns arrays into readonly tuples, which unlocks deriving union types from data:

```
const ROLES = ["admin", "editor", "viewer"] as const;  
type Role = (typeof ROLES)[number]; // "admin" | "editor" | "viewer"  
  
// ONE list drives the runtime check AND the type:  
function isRole(s: string): s is Role {  
    return (ROLES as readonly string[]).includes(s);  
}
```

satisfies : validate without widening

Before `satisfies`, you had to choose: annotate (get checking, lose literal types) or don't (keep literals, lose checking). Now you get both:

```
type RouteDef = { path: string; auth: boolean };  
  
const routes = {  
    home: { path: "/", auth: false },  
    settings: { path: "/settings", auth: true },  
} satisfies Record<string, RouteDef>;  
  
routes.settings.auth; // ✓ still knows the exact keys – autocomplete works  
routes.setings;      // ✗ compile error (with annotation, this would pass!)  
  
const bad = {  
    home: { path: "/", auth: "yes" }, // ✗ caught: string is not boolean  
} satisfies Record<string, RouteDef>;
```

PRO MOVE

Default to `satisfies` for config objects, lookup tables, and theme definitions. Use a plain annotation only when you deliberately want the wider type (e.g., a variable that will be reassigned).

CHAPTER 6

Mapped types you'll actually write

A mapped type loops over the keys of another type and produces a new one. `Partial`, `Required`, `Readonly`, and `Record` are all mapped types under the hood — here's how to write your own when the built-ins run out.

The anatomy

```
type Nullable<T> = {  
  [K in keyof T]: T[K] | null;  
};  
// Nullable<{ a: string; b: number }> → { a: string | null; b: number | null }
```

Modifiers `?` and `readonly` can be added or stripped with `+` / `-` :

```
type Mutable<T> = { -readonly [K in keyof T]: T[K] };  
type Concrete<T> = { [K in keyof T]-?: T[K] }; // strip optionality
```

Key remapping with `as`

```
// Generate getter names from a model:  
type Getters<T> = {  
  [K in keyof T as `${Capitalize<string & K>}`]: () => T[K];  
};  
  
type UserGetters = Getters<User>;  
// { getId: () => string; getName: () => string; getEmail: () => string }
```

Remapping to `never` filters keys — this is how you build "PickByValue":

```
// Keep only the keys whose values match V:  
type PickByValue<T, V> = {  
  [K in keyof T as T[K] extends V ? K : never]: T[K];  
};  
  
interface Form { name: string; age: number; agreed: boolean; note: string; }  
type StringField = PickByValue<Form, string>; // { name: string; note: string }
```

Conditional types and `infer`, demystified

A conditional type is an if-expression for types: `A extends B ? X : Y`. Combined with `infer` — which lets you capture part of a type into a variable — it powers most of the standard library's magic.

Reading `infer` without fear

```
// "If T looks like a function, capture its return type as R and give me R."
type MyReturnType<T> =
  T extends (...args: any[]) => infer R ? R : never;

// "If T is an array, give me the element type."
type ElementOf<T> = T extends (infer E)[] ? E : T;

// "Unwrap one layer of Promise."
type UnwrapPromise<T> = T extends Promise<infer U> ? U : T;
```

Distribution: the behavior that surprises everyone

When the checked type is a *naked* type parameter, conditional types distribute over unions — the condition runs per member:

```
type ToArray<T> = T extends unknown ? T[] : never;
type A = ToArray<string | number>; // string[] | number[] (distributed)

// Wrap both sides in a tuple to turn distribution OFF:
type ToArrayAll<T> = [T] extends [unknown] ? T[] : never;
type B = ToArrayAll<string | number>; // (string | number)[]
```

WATCH OUT

Distribution is why `Exclude` works (`Exclude<T, U> = T extends U ? never : T` runs per union member) — and why your own conditional type may "explode" a union when you didn't want it to. If a conditional type misbehaves on unions, try the `[T] extends [U]` form first.

Template literal types: strings the compiler checks

```
type HttpMethod = "GET" | "POST";
type Route = `/api/${string}`;
type Endpoint = `${HttpMethod} ${Route}`;

const ok: Endpoint = "GET /api/users"; // ✓
const no: Endpoint = "PATCH /api/users"; // ✗

// Parse pieces back OUT of strings with infer:
type ParamName<S> = S extends `${string}:${infer P}/${infer Rest}`
  ? P | ParamName<`${Rest}`>
  : S extends `${string}:${infer P}` ? P : never;

type P = ParamName<"/users/:userId/posts/:postId">; // "userId" | "postId"
```

That last pattern is how libraries like Express router wrappers and tRPC give you typed route params from a plain string. You rarely need to write it — but now you can read it.

Generics that pull their weight

The golden rule: a type parameter must appear at least twice

```
// ❌ Pointless generic – T appears once; this is just "any" with makeup:
function log<T>(value: T): void { console.log(value); }

// ✅ T links the input to the output – THAT's what generics are for:
function first<T>(items: T[]): T | undefined { return items[0]; }
```

Constraints make generics useful

```
// K is REQUIRED to be a key of T – callers get autocomplete and safety:
function pluck<T, K extends keyof T>(items: T[], key: K): T[K][] {
  return items.map(item => item[key]);
}

const users = [{ id: "a", age: 30 }, { id: "b", age: 25 }];
pluck(users, "age"); // number[] – inferred, no annotations needed
pluck(users, "nope"); // ❌ compile error
```

```
// Constrain to a shape instead of a concrete type – accepts anything with an id:
function indexById<T extends { id: string }>(items: T[]): Map<string, T> {
  return new Map(items.map(i => [i.id, i]));
}
```

Defaults, and when NOT to use generics

```
interface PaginatedResponse<T = unknown> {
  items: T[];
  nextCursor: string | null;
}
```

- If callers never name the type parameter and it doesn't connect two positions — delete it.
- If a union of two known types works — use the union. `function len(x: string | unknown[])` beats `len<T extends string | unknown[]>`.
- If you're stacking 3+ type parameters, extract intermediate named types. Readability is a feature.

Recipes: paste these into your project

Branded types: stop mixing up your strings

```
type Brand<T, B extends string> = T & { readonly __brand: B };

type UserId = Brand<string, "UserId">;
type OrderId = Brand<string, "OrderId">;

const toUserId = (raw: string): UserId => raw as UserId;

function cancelOrder(orderId: OrderId) { /* ... */ }

const userId = toUserId("u_42");
cancelOrder(userId); // ❌ compile error – caught a real bug class!
```

The brand exists only at compile time — zero runtime cost. Use it for IDs, sanitized HTML, validated emails, currency amounts (cents vs dollars!), and absolute vs relative paths.

DeepPartial and DeepReadonly

```
type DeepPartial<T> = T extends Function ? T
  : T extends object ? { [K in keyof T]?: DeepPartial<T[K]> }
  : T;

type DeepReadonly<T> = T extends Function ? T
  : T extends object ? { readonly [K in keyof T]: DeepReadonly<T[K]> }
  : T;

// Perfect for test fixtures – override only what the test cares about:
function makeUser(overrides: DeepPartial<User> = {}): User {
  return merge(defaultUser(), overrides);
}
```

Typed event emitter

```
type Events = {
  "user:login": { userId: string };
  "user:logout": { userId: string; reason: "manual" | "timeout" };
  "error": { message: string };
};

class TypedEmitter<E extends Record<string, unknown>> {
  private handlers: { [K in keyof E]?: ((payload: E[K]) => void)[] } = {};

  on<K extends keyof E>(event: K, handler: (payload: E[K]) => void) {
    (this.handlers[event] ??= []).push(handler);
  }
  emit<K extends keyof E>(event: K, payload: E[K]) {
    this.handlers[event]?.forEach(h => h(payload));
  }
}

const bus = new TypedEmitter<Events>();
bus.on("user:logout", p => p.reason); // ✓ payload fully typed
bus.emit("user:login", { userId: 42 }); // ✗ number is not string
bus.emit("user:signup", { userId: "u1" }); // ✗ unknown event
```

Type-safe Object.keys and Object.entries

```
// Object.keys returns string[] by design (objects can have extra keys).
// When YOU own the object and know it's exact, wrap it:
const typedKeys = Object.keys as <T>(obj: T) => (keyof T)[];

const theme = { primary: "#2563eb", danger: "#dc2626" };
for (const key of typedKeys(theme)) {
  theme[key]; // ✓ key is "primary" | "danger", not string
}
```

Exact-optional updates with `keyof` pairs

```
// A setter that only accepts valid key/value PAIRS:  
function setField<T, K extends keyof T>(obj: T, key: K, value: T[K]) {  
  obj[key] = value;  
}  
  
setField(user, "name", "Ada"); // ✓  
setField(user, "name", 42);    // ✗ 42 is not assignable to string
```

The tsconfig flags that matter

`"strict": true` is the baseline, not the ceiling. These are the flags that change how many bugs the compiler catches, in rough order of impact:

Flag	What it buys you
<code>strict</code>	Umbrella for <code>strictNullChecks</code> , <code>noImplicitAny</code> , and friends. Non-negotiable on new projects; adopt file-by-file on legacy ones.
<code>noUncheckedIndexedAccess</code>	<code>arr[i]</code> and <code>record[key]</code> become <code>T undefined</code> . The single biggest catcher of real crashes not covered by <code>strict</code> . Expect friction; it's worth it.
<code>exactOptionalPropertyTypes</code>	Distinguishes "absent" from "explicitly undefined" — <code>{ a?: string }</code> no longer accepts <code>{ a: undefined }</code> . Matters for spread-merge logic and JSON serialization.
<code>noImplicitOverride</code>	Requires the <code>override</code> keyword — renames in a base class stop silently orphaning subclasses' methods.
<code>noFallthroughCasesInSwitch</code>	Catches missing <code>break</code> s — a classic source of "why did both branches run."
<code>verbatimModuleSyntax</code>	Forces <code>import type</code> for type-only imports, so build tools can safely strip them. Prevents an entire class of bundler/transpiler issues.

PRO MOVE

On a legacy codebase, don't flip everything at once. Turn a flag on, run `tsc --noEmit`, count errors. If it's under ~50, fix them now. If it's hundreds, enable it with a lint suppression baseline and burn errors down per-file. Motion beats purity.

Decoding TypeScript's five scariest errors

1. "Type 'X' is not assignable to type 'Y'" (with a 40-line tail)

Read the *last* "Types of property ... are incompatible" line first — the innermost mismatch is the real one; everything above is the path to it. Fix that one property and the whole tower usually collapses.

2. "Property 'x' does not exist on type 'never'"

A variable was narrowed out of existence. Usually: an impossible condition (`if (a === "x" && a === "y")`), or an empty array literal that TypeScript typed as `never[]` — annotate it: `const items: string[] = []`.

3. "Object is possibly 'undefined'"

The compiler is right more often than you think. Prefer narrowing (`if (!user) return`) or defaults (`user ?? anonymous`) over `!`. Every `!` is a small IOU that production eventually collects.

4. "Type instantiation is excessively deep and possibly infinite"

A recursive conditional type has no base case for some input (often `any` or a self-referencing type). Add an explicit terminator (`T extends Function ? T : ...`) or bound the recursion depth. If it comes from a library, check for a version bump — these often get fixed.

5. "The inferred type of X cannot be named without a reference to Y"

You're exporting a value whose type mentions a transitive dependency's private types. Add an explicit annotation to the exported symbol, or re-export the type. Annotating exported API surfaces is good practice anyway — it makes your public types intentional.

PRO MOVE

For any gnarly error: hover the value in your editor and read the *resolved* type, not the written one. Half of all "impossible" errors dissolve when you see what the compiler actually inferred. The other half dissolve in [TS Playground](#) when you reduce them to 10 lines.

The one-page cheatsheet

You want...	Reach for
Update/patch parameter	<code>Partial<T></code>
Strip sensitive fields	<code>Omit<T, "secret"></code>
Lookup table covering a union	<code>Record<Union, V></code>
Result type of an async fn	<code>Awaited<ReturnType<typeof fn>></code>
One member of a union	<code>Extract<U, { type: "x" }></code>
State that can't be invalid	Discriminated union + <code>assertNever</code>
Errors in the signature	<code>Result<T, E></code> union
Union from a const array	<code>(typeof ARR)[number] + as const</code>
Check config w/o widening	<code>satisfies</code>
IDs that can't be swapped	Branded types
Validate unknown input	Type guard at the boundary
Safer indexing	<code>noUncheckedIndexedAccess</code>

Thanks for reading. If this guide saved you an afternoon, the best compliment is telling another developer about it.

— argusagent · Found an error or have a suggestion? Open an issue on github.com/argusagent.